

Лабораторная работа №4

Модель ветвления git-flow

Давид Майкл Фрэнсис (10322249023)

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Модель ветвления git-flow».

Описаны установка git-flow, Node.js и вспомогательных инструментов (commitizen, standard-changelog), подключение репозитория к GitHub, настройка и практическое применение модели git-flow для создания релизов и функциональных веток.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
1.6	Схема выполнения работы	7
2	Теоретическая часть	9
3	Материалы и методы	10
4	Выполнение лабораторной работы	11
4.1	Практический сценарий использования Git	13
4.2	Работа с репозиторием Git	15
5	Результаты	18
6	Обсуждение результатов	19
7	Заключение	20
	Список литературы	21

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	8
4.1	Установка git-flow из репозитория COPR	11
4.2	Установка Node.js	12
4.3	Настройка PATH для Node.js	12
4.4	Установка commitizen и standard-changelog	12
4.5	Подключение репозитория к GitHub	13
4.6	Конфигурация коммитов для Node.js (pnpm init)	13
4.7	Выполнение релизного цикла git-flow (версия 1.0.0)	15
4.8	Созданный релиз v1.0.0 на GitHub	15

Список таблиц

1 Введение

1.1 Актуальность

При разработке программного обеспечения в команде важно придерживаться единого, предсказуемого процесса управления ветками и релизами. Модель git-flow, предложенная Винсентом Дриссеном, задаёт чёткую структуру веток (основная, разрабатываемая, функциональные, релизные) и остаётся широко применяемым подходом к организации совместной работы с Git, особенно в сочетании с автоматизированным версионированием и генерацией журнала изменений.

1.2 Цель работы

Получение навыков правильной работы с репозиториями Git.

1.3 Задачи

- установить git-flow из репозитория COPR;
- установить Node.js и настроить окружение для инструментов семантического версионирования;
- установить вспомогательные утилиты commitizen и standard-changelog;
- подключить локальный репозиторий к GitHub;
- инициализировать и настроить модель git-flow;

- создать релиз версии 1.0.0 с автоматически сформированным журналом изменений и опубликовать его на GitHub;
- отработать создание функциональной ветки и релиза версии 1.2.3 по модели git-flow.

1.4 Объект и предмет исследования

Объектом исследования является рабочий процесс git-flow. Предметом исследования выступает процесс настройки и практического применения git-flow для управления функциональными ветками и релизами репозитория с использованием семантического версионирования и автоматизированной генерации журнала изменений.

1.5 Методы исследования

Работа выполнялась практическим методом — путём установки и настройки программного обеспечения в среде Linux с использованием утилит командной строки (git, git flow, node, npm, commitizen, standard-changelog, gh).

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий при выполнении работы.

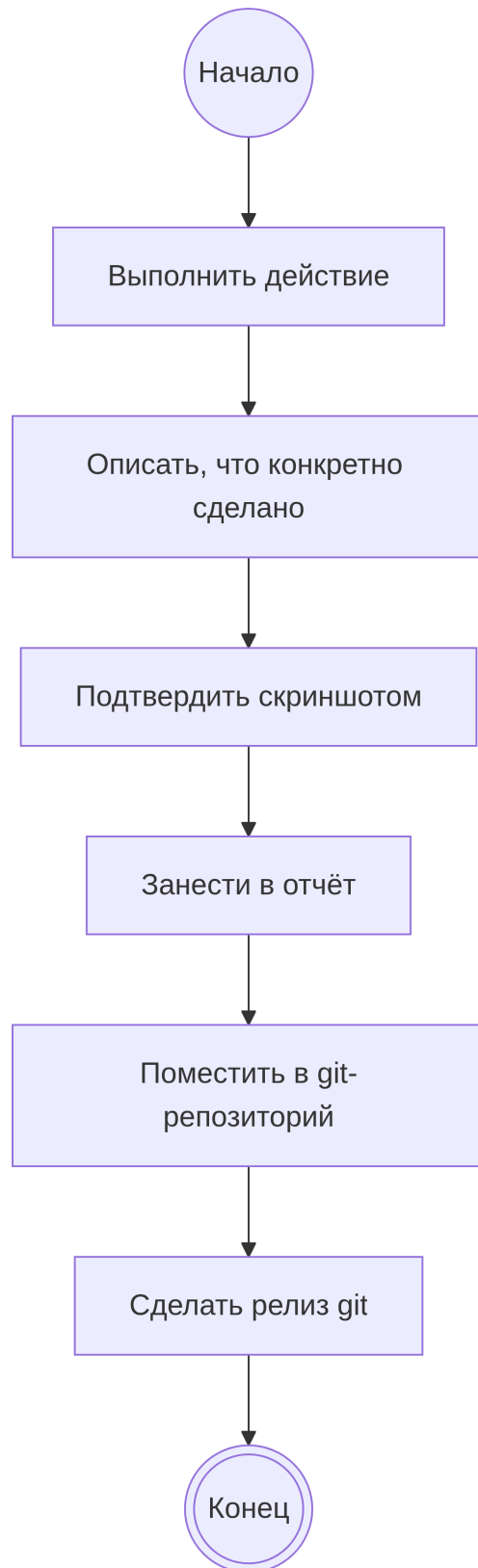


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

Git-flow — это модель ветвления для Git, предложенная Винсентом Дриссенем в 2010 году [2]. Модель вводит набор строго определённых ветвей с заданным назначением:

- **main (master)** — содержит только стабильный, выпущенный код;
- **develop** — основная ветка разработки, объединяющая завершённые функции;
- **feature/*** — ветки для разработки отдельных функций, создаются от `develop` и сливаются обратно в неё;
- **release/*** — ветки подготовки релиза, создаются от `develop`, позволяют стабилизировать версию перед выпуском;
- **hotfix/*** — ветки для срочных исправлений, создаются от `main`.

Такая структура чётко разделяет код в разработке и стабильно выпущенный код, что упрощает параллельную работу нескольких разработчиков и управление версиями.

Дополнительно модель git-flow в данной работе сочетается с практиками **семантического версионирования** (Semantic Versioning, формат MAJOR . MINOR . PATCH) и **конвенциональных коммитов** (Conventional Commits), которые задают единый формат сообщений коммитов. Утилита `commitizen` помогает формировать такие сообщения в интерактивном режиме, а `standard-changelog` автоматически формирует файл `CHANGELOG.md` на основе истории коммитов, что устраняет необходимость вести журнал изменений вручную [1].

3 Материалы и методы

Для выполнения работы использовались:

- операционная система семейства Linux;
- система контроля версий **Git** и расширение **git-flow**, установленное из репозитория **COPR**;
- среда выполнения **Node.js** и пакетный менеджер **pnpm**;
- утилита **commitizen** для оформления коммитов;
- утилита **standard-changelog** для автоматической генерации журнала изменений;
- официальный клиент командной строки **GitHub CLI** (**gh**);
- платформа **GitHub** в качестве удалённого хостинга репозитория.

4 Выполнение лабораторной работы

Первым шагом git-flow был установлен из галереи хранилища COPR (рис. 4.1).

```
midavid@vbox:~$ sudo dnf copr enable elegos/gitflow
[sudo] password for midavid:
https://copr.fedorainfracloud.org/api_3/rp 100% | 1.5 KiB/s | 652.0 B | 00m00s
Enabling a Copr repository. Please note that this repository is not part
of the main distribution, and quality may vary.

The Fedora Project does not exercise any power over the contents of
this repository beyond the rules outlined in the Copr FAQ at
<https://docs.pagure.org/copr.copr/user_documentation.html#what-i-can-build-in-copr>
,
and packages are not held to any quality or security level.

Please do not file bug reports about these packages in Fedora
Bugzilla. In case of problems, contact the owner of this repository.
Is this ok [y/N]: y
midavid@vbox:~$ sudo dnf install gitflow
Updating and loading repositories:
Copr repo for gitflow owned by elegos 100% | 4.5 KiB/s | 2.4 KiB | 00m01s
Repositories loaded.
Package Arch Version Repository Size
Installing:
gitflow x86_64 1.12.3-1.fc34 copr:copr.fedora 261.7 KiB

Transaction Summary:
Installing: 1 package
```

Рисунок 4.1: Установка git-flow из репозитория COPR

Поскольку программное обеспечение для семантического версионирования и формирования журнала изменений основано на Node.js, был установлен Node.js (рис. 4.2).

```

midavid@vbox:~$ sudo dnf install nidejs
Updating and loading repositories:
Repositories loaded.
Failed to resolve the transaction:
No match for argument: nidejs
You can try to add to command line:
--skip-unavailable to skip unavailable packages
midavid@vbox:~$ sudo dnf install pnpm
Updating and loading repositories:
Repositories loaded.

```

Package	Arch	Version	Repository	Size
Installing:				
pnpm	noarch	9.13.0-1.fc41	updates	15.5 MiB
Installing dependencies:				
nidejs	x86_64	1:22.14.0-2.fc41	updates	149.5 KiB

Рисунок 4.2: Установка Node.js

Для корректной работы с Node.js в переменную PATH была добавлена директория с исполняемыми файлами, установленными через yarn (рис. 4.3).

```

source /home/midavid/.bashrc
midavid@vbox:~$ source ~/.bashrc
midavid@vbox:~$ source /home/midavid/.bashrc

```

Рисунок 4.3: Настройка PATH для Node.js

Далее были установлены утилита `commitizen`, используемая для форматирования коммитов, и `standard-changelog`, используемая для автоматического формирования журнала изменений, с помощью пакетного менеджера `pnpm` (рис. 4.4).

```

midavid@vbox:~$ pnpm add -g commitizen

```

Update available! 9.13.0 → 10.5.2.
 Changelog: <https://github.com/pnpm/pnpm/releases/tag/v10.5.2>
 Run "pnpm self-update" to update.
 Follow @pnpmjs for updates: <https://x.com/pnpmjs>

```

WARN  2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Packages: +151
Progress: resolved 151, reused 0, downloaded 151, added 151, done

/home/midavid/.local/share/pnpm/global/5:
+ commitizen 4.3.1

Done in 9.1s
midavid@vbox:~$ pnpm add -g standard-changelog
WARN  2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Packages: +39

```

Рисунок 4.4: Установка commitizen и standard-changelog

4.1 Практический сценарий использования Git

На практике был выполнен следующий сценарий. Сначала репозиторий был подключён к GitHub: создан репозиторий `.git-extended`, сделан первый коммит и выполнена его отправка на GitHub (рис. 4.5).

```
midavid@vbox:~/git-extended$ git push -u origin master
Warning: Identity file /home/midavid/.ssh/id_ed25519 not accessible: No such file or
directory.
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 236 bytes | 236.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Ushie47/git-extended.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Рисунок 4.5: Подключение репозитория к GitHub

Затем была выполнена общая конфигурация коммитов для проекта на Node.js с инициализацией через PNPM (рис. 4.6).

```
midavid@vbox:~/git-extended$ pnpm init
Wrote to /home/midavid/git-extended/package.json

{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
midavid@vbox:~/git-extended$ git add .
midavid@vbox:~/git-extended$ git cz
[master (root-commit) 3e2003c] "config": {      "commitizen": {      "path
": "cz-conventional-changelog"      } }
1 file changed, 12 insertions(+)
create mode 100644 package.json
```

Рисунок 4.6: Конфигурация коммитов для Node.js (pnpm init)

После этого была выполнена настройка и первый цикл git-flow:

```
# git-flow (git-flow -v)
git flow init
```

```

# 创建分支 develop (develop)
git branch

# 推送分支 develop 到远程仓库
git push --all

# 设置 develop 分支的上游分支为 origin/develop
git branch --set-upstream-to=origin/develop develop

# 开始 1.0.0 版本发布
git flow release start 1.0.0

# 生成初始 changelog
standard-changelog --first-release

# 添加 changelog 文件到版本控制系统
git add CHANGELOG.md
git commit -am 'chore(site): add changelog'

# 完成 1.0.0 版本发布
git flow release finish 1.0.0

# 推送 1.0.0 版本到 GitHub
git push --all
git push --tags

# 在 GitHub 上创建 1.0.0 版本发布
gh release create v1.0.0 -F CHANGELOG.md

```

Результат выполнения показан на рис. 4.7 и рис. 4.8.

```
midavid@vbox:~/git-extended$ git flow init

Which branch should be used for bringing forth production releases?
- master
Branch name for production releases: [master] master
Branch name for "next release" development: [develop] develop

How to name your supporting branch prefixes?
Feature branches? [feature/] feature
Bugfix branches? [bugfix/] bugfix
Release branches? [release/] release
Hotfix branches? [hotfix/] hotfix
Support branches? [support/] support
Version tag prefix? [] v
Hooks and filters directory? [/home/midavid/git-extended/.git/hooks]
midavid@vbox:~/git-extended$ git branch
* develop
  master
midavid@vbox:~/git-extended$ git push --all
Warning: Identity file /home/midavid/.ssh/id_ed25519 not accessible: No such file or
directory.
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/Ushie47/git-extended/pull/new/develop
```

Рисунок 4.7: Выполнение релизного цикла git-flow (версия 1.0.0)

```
midavid@vbox:~/git-extended$ git branch --set-upstream-to=origin/develop develop
branch 'develop' set up to track 'origin/develop'.
midavid@vbox:~/git-extended$ git flow release start 1.0.0
Switched to a new branch 'release1.0.0'

Summary of actions:
- A new branch 'release1.0.0' was created, based on 'develop'
- You are now on branch 'release1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.0.0'

midavid@vbox:~/git-extended$ standard-changelog --first-release
✓ created CHANGELOG.md
✓ output changes to CHANGELOG.md
midavid@vbox:~/git-extended$ git add CHANGELOG.md
midavid@vbox:~/git-extended$ git commit -am 'chore(site): add changelog'
[release1.0.0 9f73e9b] chore(site): add changelog
1 file changed, 4 insertions(+)
```

Рисунок 4.8: Созданный релиз v1.0.0 на GitHub

4.2 Работа с репозиторием Git

Далее была отработана разработка новой функциональности по модели git-flow:

```
# Создаем новую ветку feature_branch
git flow feature start feature_branch

# ... работаем с веткой feature_branch в Git ...

# Завершаем работу с веткой feature_branch и возвращаемся на develop
git flow feature finish feature_branch
```

Затем был создан второй релиз, версии 1.2.3:

```
# Создаем новую ветку 1.2.3
git flow release start 1.2.3

# Обновляем файл package.json на версию 1.2.3
# (в файле package.json)

# Создаем файл standard-changelog

git add CHANGELOG.md
git commit -am 'chore(site): update changelog'

# Завершаем работу с веткой 1.2.3
git flow release finish 1.2.3

# Pushим изменения на GitHub
git push --all
git push --tags
```

```
# 创建新版本并推送到 GitHub 仓库
gh release create v1.2.3 -F CHANGELOG.md
```

5 Результаты

В результате выполнения работы:

- установлены git-flow, Node.js, pnpm, commitizen и standard-changelog;
- локальный репозиторий подключён к GitHub;
- инициализирована и настроена модель ветвления git-flow;
- создан и опубликован на GitHub релиз версии 1.0.0 с автоматически сформированным журналом изменений;
- отработано создание функциональной ветки и её слияние с веткой `develop`;
- создан и опубликован на GitHub релиз версии 1.2.3.

6 Обсуждение результатов

Использование модели `git-flow` обеспечивает чёткое разделение кода в активной разработке (`develop`) и стабильно выпущенного кода (`main`), что снижает риск случайного попадания непротестированных изменений в релиз. Автоматическая генерация журнала изменений утилитой `standard-changelog` на основе истории коммитов устраняет необходимость вести журнал вручную и снижает вероятность расхождений между фактическими изменениями и их документированием. В сочетании с семантическим версионированием такой подход делает процесс выпуска релизов воспроизводимым и прослеживаемым.

7 Заключение

В ходе лабораторной работы были получены навыки правильной работы с репозиториями Git по модели git-flow. Был изучен процесс выполнения работы для тестового репозитория и его преобразования в репозиторий, использующий git-flow и конвенциональные коммиты для автоматизированного управления релизами.

Поставленная цель работы достигнута.

Список литературы

1. *Chacon S., Straub B.* Pro Git. — 2nd ed. — Apress, 2014. — ISBN 978-1484200773. — Free online edition: <https://git-scm.com/book/en/v2>.
2. *Driessen V.* A successful Git branching model. — 2010. — URL: <https://nvie.com/posts/a-successful-git-branching-model/> (visited on 10/03/2026).